

COMPLEXITY ANALYSIS REVISITED

Analyze the running time of mystery

```
void mystery(int n) {  
    for (int i = 0; i < n; i++){  
        cout << i; // statement A  
        for (int j = i; j < n; j++){  
            cout << j; // statement B  
        }  
    }  
}
```

$O(1)$

$O(1)$

Which statement dominates the running time of mystery?

A. Statement A **B. Statement B** C. Both matter equally

General Insight

```
void mystery(int n) {  
    for (int i = 0; i < n; i++){  
        cout << i; // A: Outer loop operation  
        for (int j = i; j < n; j++){  
            cout << j; // B: Inner loop operation  
        }  
    }  
}
```

When two operations have the same complexity, the one that runs more times dominates.

So total cost = count total executions of the dominant operation.

BFS: Running Time Complexity

$$G = (V, E)$$

$$T_{BFS}(n, m)$$

$\uparrow$ no. of vertices $\nwarrow$ no. of edges

Algo exploreBFS (Graph G, vertex s):

- T_{init}*
- Mark all the vertices as "not visited"
 - Mark **s** as visited
 - push **s** into a queue
 - while the queue is not empty:
 - A*: • pop the vertex **u** from the front of the queue *T_A*
 - for each of **u**'s neighbor (**v**)
 - B*: • If **v** has not yet been visited: *T_B*
 - Mark **v** as visited
 - C*: • Push **v** in the queue *T_C*

$$\begin{aligned}
 &= T_{init}(n, m) + T_{while}(n, m) \\
 &= O(n) + T_A + T_B + T_C \\
 &= O(n) + T_B \quad (B \text{ dominates})
 \end{aligned}$$

Which statement dominates the running time of exploreBFS?

A, B, C are all O(1) per execution

B is inside the inner loop.

$\rightarrow T_B > T_A$ (same reason as mystery C)

C is only run if B evaluates to True

$\rightarrow T_B > T_C$

$\dots \rightarrow B \text{ dominates.}$

Fact: $T_A = T_C < T_B$

(every vertex is pushed once, so it can only be popped once. Total push/pop = n)

Total cost of while loop = Total executions of neighbor check (T_B)

$$T_B = \sum_{u \in V} \deg(u) = 2 \cdot m$$

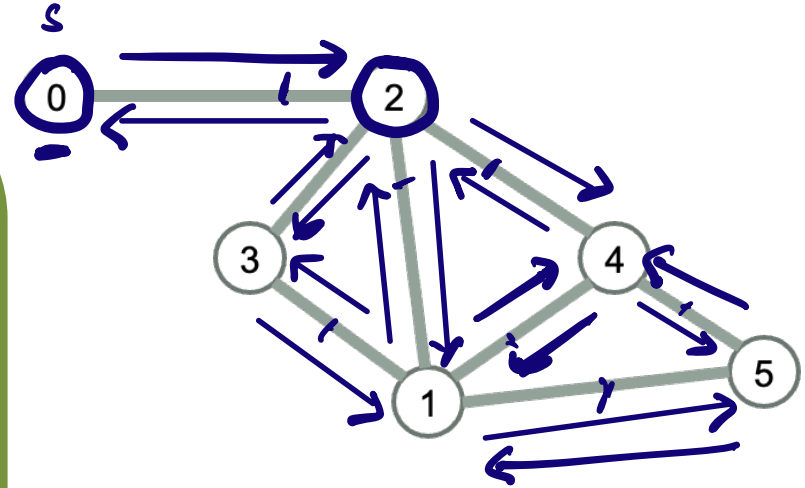
Algo exploreBFS (Graph G, vertex s):

- Mark all the vertices as “not visited”
- Mark **s** as visited
- push **s** into a queue
- while the queue is not empty:
 - pop the vertex **u** from the front of the queue
 - for each of **u**'s neighbor (**v**)

T_B $\rightarrow$ If **v** has not yet been visited:

- Mark **v** as visited
- Push **v** in the queue

$\deg(u)$ = no. of neighbors



$$\begin{aligned} T_B &= 1 + 4 + 2 + 4 + 3 + 2 \\ &= 16 \\ &= 2 \cdot 8 \\ &= 2m \end{aligned}$$

$$T(n, m) = T_{init} + T_B = O(n) + O(2 \cdot m) = O(n) + O(m) = O(n + m)$$

BFS: Space Complexity

What is the Big -O auxiliary space complexity of exploreBFS?

- A. $O(n)$
- B. $O(m)$
- C. $O(n + m)$
- D. $O(n^2)$
- E. None of the above

n: number of vertices

m: number of edges

Two extra data structures: (1) Vector of visited vertices (at most size n)
(2) Queue. (at most size n)

$$S(n) = O(n)$$

- Auxiliary Space complexity: Additional space usage (not including input and output)

exploreDFS: Time Complexity

exploreDFS(v, visited):

A: visited[v] = true

For each edge (v,w):

B: If not w.visited

C: → exploreDFS(w, visited)

exploreDFS(Graph G, vertex s):

T_{init} Mark all vertices as "not visited" $O(n)$

exploreDFS(s, visited)

T_{explore} →

T_C : No. of recursive calls.
exploreDFS is called once per vertex
n times at most = $O(n)$

T_A : $O(1)$ per executions
 $O(m)$ overall

T_B : $O(m)$ { same reason as before }

$$\begin{aligned} T_{\text{explore}} &= T_A + \underline{T_B} + \underline{T_C} \\ &= O(n) + O(m) + O(n) \\ &\geq O(n+m) \end{aligned}$$

exploreDFS: Space Complexity

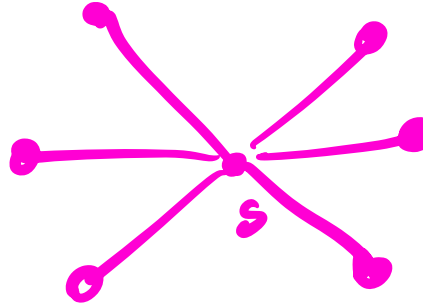
exploreDFS(v, visited):

visited[v] = true

For each edge (v,w):

If not w.visited

exploreDFS(w, visited)

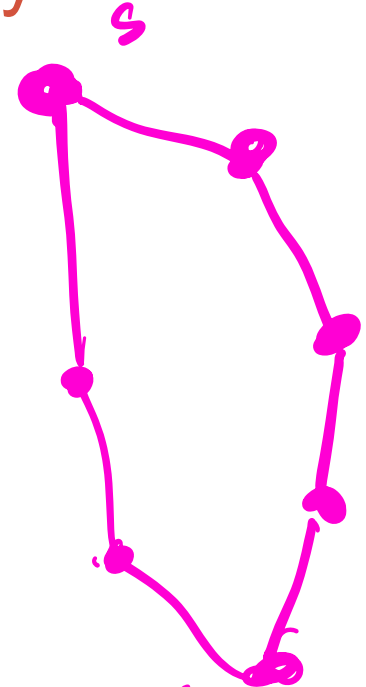


$O(1)$

exploreDFS(Graph G, vertex s):

Mark all vertices as "not visited"

exploreDFS(s, visited)



$O(n)$

Worst case depth of
recursion (depth of the stack)
= $O(n)$

What is the running time complexity for a set with n keys?

```
void printSetValues(const std::set<int>& s){  
    for (int value : s) {  
        std::cout << value << " ";  
    }  
}
```

Handwritten notes for the code above:

- ↙ n keys
- get min = $O(\log(n))$
- for(auto it = s.begin(); it != s.end(); it++) {
 cout << *it;

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$ (naive analysis)

n calls to successor.

Successor is worst case $O(\log n)$

Overall $O(n \log n)$

What is the output of this code for the given BST?

```
void printSetValues(const std::set<int>& s){  
    for (int value : s) {  
        std::cout << value << " ";  
    }  
}
```

$O(n)$
s.begin
1, 2, 4, 6, 7, 8

(1) no. of edges, $m = n - 1$, why?

Each vertex (node) has exactly one parent except the root which has no parent.

There are n vertices, $n - 1$ edges total.

(2) Every edge (u, v) is traversed exactly twice - once going down into, once going back up when leaving v .

Total edge traversals is $2 \cdot (n - 1) = O(n)$

